EDA Course Project - Police Shootings Dataset - Shaurya Garg 23BDS0161

1. Import libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats
from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from google.colab import files

pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)
sns.set_theme(style="whitegrid")

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

2. Load the dataset

```
# Upload shootings.csv from your computer.
uploaded = files.upload()

# Use the first uploaded CSV file.
csv_files = [name for name in uploaded.keys() if name.lower().endswith(".csv")]
if not csv_files:
    raise ValueError("Please upload a CSV file, preferably shootings.csv.")

file_name = csv_files[0]
df = pd.read_csv(file_name)

print("Loaded file:", file_name)
print("Dataset shape:", df.shape)
df.head()
```

Choose Files shootings.csv

shootings.csv(text/csv) - 502983 bytes, last modified: 9/13/2026 - 100% done

```
Saving shootings.csv to shootings.csv
Loaded file: shootings.csv
Dataset shape: (4895, 15)
```

	id	name	date	manner_of_death	armed	age	gender	race	city	state	signs_of_mental_illness	threat_level
0	3	Tim Elliot	02/01/15	shot	gun	53.0	M	Asian	Shelton	WA	True	attack
1	4	Lewis Lee Lembke	02/01/15	shot	gun	47.0	M	White	Aloha	OR	False	attack
2	5	John Paul Quintero	03/01/15	shot and Tasered	unarmed	23.0	M	Hispanic	Wichita	KS	False	other

Next steps: Generate code with df | New interactive sheet

3. Initial inspection

```
print("First 5 rows:")
display(df.head())

print("\\nLast 5 rows:")
display(df.tail())

print("\\nRandom sample:")
```

```
print("\\nRandom sample:")
display(df.sample(min(5, len(df)), random_state=123))

print("\\nNumber of rows and columns:")
print(df.shape)

print("\\nColumn names:")
print(df.columns.tolist())

print("\\nData types and non-null counts:")
df.info()

print("\\nStatistical summary:")
display(df.describe(include="all").T)
```

First 5 rows:

	id	name	date	manner_of_death	armed	age	gender	race	city	state	signs_of_mental_illness	threat_level
0	3	Tim Elliot	02/01/15	shot	gun	53.0	M	Asian	Shelton	WA	True	attack
1	4	Lewis Lee Lembke	02/01/15	shot	gun	47.0	M	White	Aloha	OR	False	attack
2	5	John Paul Quintero	03/01/15	shot and Tasered	unarmed	23.0	M	Hispanic	Wichita	KS	False	other
3	8	Matthew Hoffman	04/01/15	shot	toy gun	32.0	M	White	San Francisco	CA	True	attack

4. Check missing values

```
missing_by_column = df.isna().sum().sort_values(ascending=False)
missing_percent = (df.isna().mean() * 100).sort_values(ascending=False)

missing_summary = pd.DataFrame({
    "Missing Count": missing_by_column,
    "Missing Percent": missing_percent.round(2)
})
display(missing_summary)

print("Total missing values:", int(df.isna().sum().sum()))
print("Rows containing at least one missing value:", int(df.isna().any(axis=1).sum()))
print("Rows with no missing values:", int(df.notna().all(axis=1).sum()))

# Display a few rows that contain missing values.
display(df[df.isna().any(axis=1)].head(10))
```

	Missing Count	Missing Percent
manner_of_death	3	0.06
arms_category	2	0.04
race	2	0.04
date	0	0.00
armed	0	0.00
name	0	0.00
id	0	0.00
gender	0	0.00
age	0	0.00
state	0	0.00
city	0	0.00
signs_of_mental_illness	0	0.00
threat_level	0	0.00
flee	0	0.00
body_camera	0	0.00

	id	name	date	manner_of_death	armed	age	gender	race	city	state	signs_of_mental_illness	threat_level
21	42	Tommy Smith	11/01/15	shot	gun	39.0	M	White	Arcola	IL	True	
22	43	Brian Barbosa	11/01/15	shot	gun	23.0	M	Hispanic	South Gate	CA	False	
117	130	Betty Sexton	17/02/15	shot	gun	43.0	F	NaN	Gaston County	NC	False	
118	159	Michael Ireland	17/02/15	shot	unarmed	31.0	M	NaN	Springfield	MO	False	

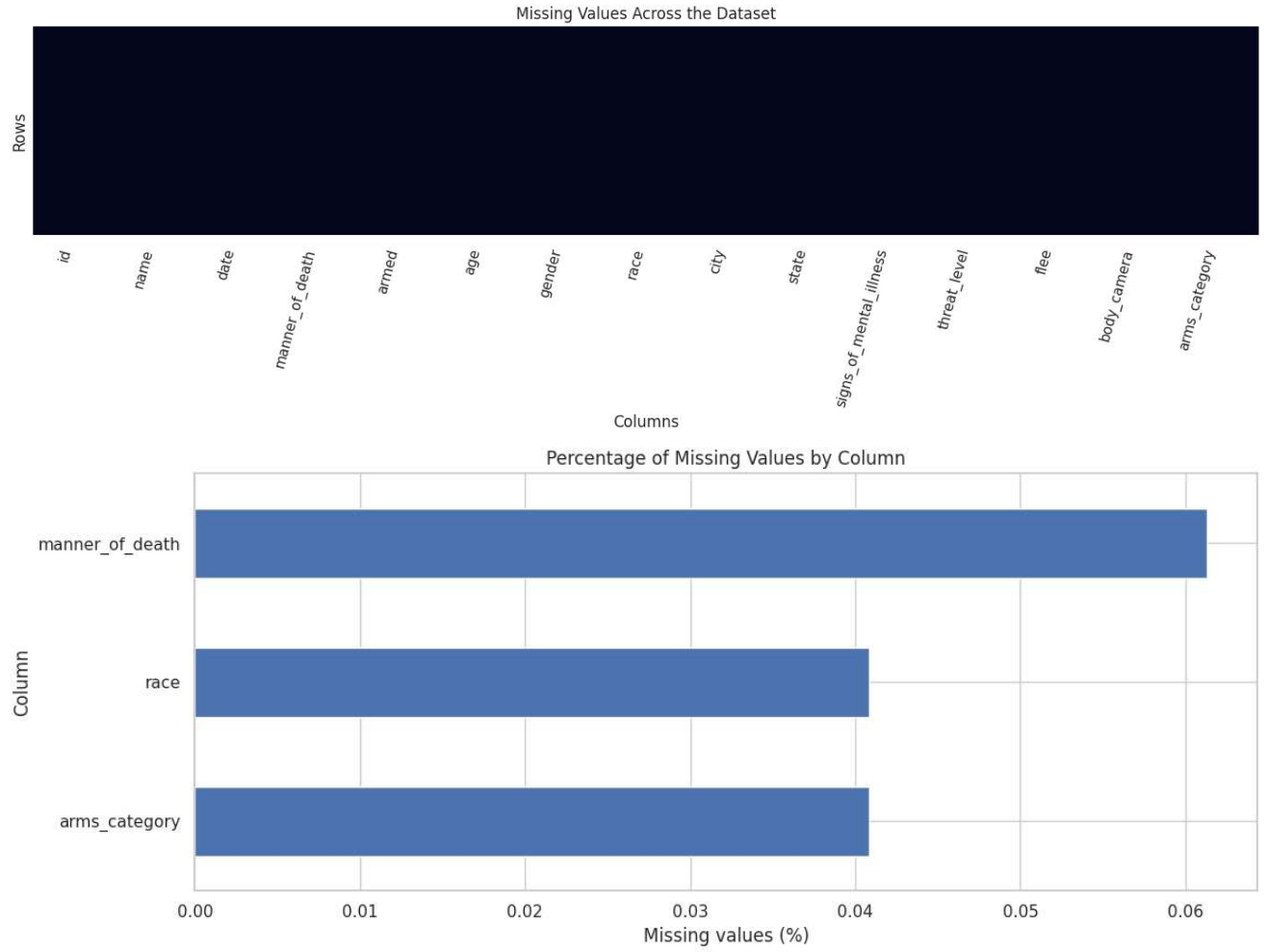
Next steps: Generate code with missing_summary | New interactive sheet

4A. Visualize missing values

	count	unique	top	freq	mean	std	min	25%	50%	75%	max
id	4895.0	NaN	NaN	NaN	2902.148519	1683.46791	3.0	1441.5	2847.0	4352.5	5925.0

```
plt.figure(figsize=(14, 5))
sns.heatmap(df.isna(), cbar=False, yticklabels=False)
plt.title("Missing Values Across the Dataset")
plt.xlabel("Columns")
plt.ylabel("Rows")
plt.xticks(rotation=75, ha="right")
plt.tight_layout()
plt.show()

plt.figure(figsize=(12, 5))
missing_percent[missing_percent > 0].sort_values().plot(kind="barh")
plt.title("Percentage of Missing Values by Column")
plt.xlabel("Missing values (%)")
plt.ylabel("Column")
plt.tight_layout()
plt.show()
```



5. Missing-value handling methods

The following cells demonstrate multiple methods from the assignments. Each method starts from a fresh copy of the original data so that the results do not overwrite one another.

```
# Method 1: Remove rows containing at least one missing value.
df_drop_rows = df.dropna()
print("Original shape:", df.shape)
print("Shape after dropping incomplete rows:", df_drop_rows.shape)

# Method 2: Remove columns with more than 70% missing values.
missing_pct_original = df.isna().mean() * 100
columns_over_70 = missing_pct_original[missing_pct_original > 70].index.tolist()
df_drop_columns = df.drop(columns=columns_over_70)

print("Columns removed (>70% missing):", columns_over_70)
print("Shape after removing those columns:", df_drop_columns.shape)
```

```
Original shape: (4895, 15)
Shape after dropping incomplete rows: (4888, 15)
Columns removed (>70% missing): []
Shape after removing those columns: (4895, 15)
```

```
# Method 3: Mean imputation for numeric columns.
df_mean_imputed = df.copy()
numeric_cols = df_mean_imputed.select_dtypes(include=np.number).columns
df_mean_imputed[numeric_cols] = df_mean_imputed[numeric_cols].fillna(
    df_mean_imputed[numeric_cols].mean()
)
print("Missing values remaining after numeric mean imputation:",
      int(df_mean_imputed[numeric_cols].isna().sum().sum()))

# Method 4: Median imputation for numeric columns.
df_median_imputed = df.copy()
numeric_cols_median = df_median_imputed.select_dtypes(include=np.number).columns
df_median_imputed[numeric_cols_median] = df_median_imputed[numeric_cols_median].fillna(
    df_median_imputed[numeric_cols_median].median()
)
print("Missing values remaining after numeric median imputation:",
      int(df_median_imputed[numeric_cols_median].isna().sum().sum()))
```

```
Missing values remaining after numeric mean imputation: 0
Missing values remaining after numeric median imputation: 0
```

```
# Method 5: Mode imputation for categorical/text columns.
df_mode_imputed = df.copy()
categorical_cols = df_mode_imputed.select_dtypes(include=["object", "category", "bool"]).columns

for col in categorical_cols:
    modes = df_mode_imputed[col].mode(dropna=True)
    if not modes.empty:
        df_mode_imputed[col] = df_mode_imputed[col].fillna(modes.iloc[0])

print("Missing values remaining in categorical columns:",
      int(df_mode_imputed[categorical_cols].isna().sum().sum()))

# Method 6: Forward fill and backward fill, shown separately.
df_forward_filled = df.copy().ffill()
df_backward_filled = df.copy().bfill()

print("Missing values after forward fill:", int(df_forward_filled.isna().sum().sum()))
print("Missing values after backward fill:", int(df_backward_filled.isna().sum().sum()))

# Method 7: Replace remaining missing values with zero, on a separate copy.
df_missing_zero = df.copy().fillna(0)
print("Missing values after replacing with zero:", int(df_missing_zero.isna().sum().sum()))
```

```
Missing values remaining in categorical columns: 0
Missing values after forward fill: 0
Missing values after backward fill: 0
Missing values after replacing with zero: 0
```

Note: Mean/median imputation is mainly appropriate for numeric measurements. Mode imputation is often used for categorical values. Forward/backward fill is more appropriate for ordered or time-series data; this dataset represents individual incidents, so carrying values between rows may not be meaningful. Replacing all missing values with zero can create misleading values, especially for age or categorical columns.

6. Duplicate records

```
duplicate_count = int(df.duplicated().sum())
print("Number of duplicate rows:", duplicate_count)

df_no_duplicates = df.drop_duplicates().copy()
print("Shape before removing duplicates:", df.shape)
print("Shape after removing duplicates:", df_no_duplicates.shape)
```

```
Number of duplicate rows: 0
Shape before removing duplicates: (4895, 15)
Shape after removing duplicates: (4895, 15)
```

7. Clean column names and standardize text

```
df_clean = df_no_duplicates.copy()

# Remove leading/trailing spaces in column names.
df_clean.columns = df_clean.columns.str.strip()

# Normalize text values in object columns: trim extra spaces.
object_cols = df_clean.select_dtypes(include="object").columns
for col in object_cols:
    df_clean[col] = df_clean[col].map(
        lambda value: value.strip() if isinstance(value, str) else value
    )

# Convert text values to lowercase for consistency.
# This changes capitalization only; it does not merge different categories.
for col in object_cols:
    df_clean[col] = df_clean[col].map(
        lambda value: value.lower() if isinstance(value, str) else value
    )

print("Cleaned column names:")
print(df_clean.columns.tolist())
display(df_clean.head())
```

```
Cleaned column names:
['id', 'name', 'date', 'manner_of_death', 'armed', 'age', 'gender', 'race', 'city', 'state', 'signs_of_mental_illness', 'threat_
```

	id	name	date	manner_of_death	armed	age	gender	race	city	state	signs_of_mental_illness	threat_level
0	3	tim elliot	02/01/15	shot	gun	53.0	m	asian	shelton	wa	True	attack
1	4	lewis lee lembke	02/01/15	shot	gun	47.0	m	white	aloha	or	False	attack
2	5	john paul quintero	03/01/15	shot and tasered	unarmed	23.0	m	hispanic	wichita	ks	False	other

8. Convert data types

```
# Convert columns when they exist in the uploaded dataset.
if "id" in df_clean.columns:
    df_clean["id"] = pd.to_numeric(df_clean["id"], errors="coerce")

if "age" in df_clean.columns:
    df_clean["age"] = pd.to_numeric(df_clean["age"], errors="coerce")

# The examples in the assignments use dates like 02/01/15 (day/month/year).
# Try that format first, then use a general parser for values that did not parse.
if "date" in df_clean.columns:
    parsed_date = pd.to_datetime(
        df_clean["date"], format="%d/%m/%y", errors="coerce"
    )
    remaining = parsed_date.isna() & df_clean["date"].notna()
    if remaining.any():
        parsed_date.loc[remaining] = pd.to_datetime(
            df_clean.loc[remaining, "date"], errors="coerce", dayfirst=True
        )
```

```
    df_clean["date"] = parsed_date

# Convert common text columns to category dtype where present.
category_candidates = [
    "manner_of_death", "armed", "gender", "race", "city", "state",
    "signs_of_mental_illness", "threat_level", "flee",
    "body_camera", "arms_category"
]
for col in category_candidates:
    if col in df_clean.columns:
        df_clean[col] = df_clean[col].astype("category")

# Rename age to Age if present, following the assignment's rename example.
if "age" in df_clean.columns:
    df_clean = df_clean.rename(columns={"age": "Age"})

# The source dataset has a 'date' column rather than a 'year' column.
# Derive Year from date instead of trying to rename a nonexistent column.
if "date" in df_clean.columns and pd.api.types.is_datetime64_any_dtype(df_clean["date"]):
    df_clean["Year"] = df_clean["date"].dt.year

df_clean.info()
display(df_clean.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4895 entries, 0 to 4894
Data columns (total 16 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   id                       4895 non-null   int64
 1   name                     4895 non-null   object
 2   date                     4895 non-null   datetime64[ns]
 3   manner_of_death          4892 non-null   category
 4   armed                    4895 non-null   category
 5   Age                      4895 non-null   float64
 6   gender                   4895 non-null   category
 7   race                     4893 non-null   category
 8   city                     4895 non-null   category
 9   state                    4895 non-null   category
 10  signs_of_mental_illness  4895 non-null   category
 11  threat_level             4895 non-null   category
 12  flee                     4895 non-null   category
 13  body_camera              4895 non-null   category
 14  arms_category            4893 non-null   category
 15  Year                     4895 non-null   int32
dtypes: category(11), datetime64[ns](1), float64(1), int32(1), int64(1), object(1)
memory usage: 318.6+ KB
```

	id	name	date	manner_of_death	armed	Age	gender	race	city	state	signs_of_mental_illness	threat_level	f
0	3	tim elliot	2015-01-02	shot	gun	53.0	m	asian	shelton	wa	True	attack	flee
1	4	lewis lee lembke	2015-01-02	shot	gun	47.0	m	white	aloha	or	False	attack	flee
2	5	john paul quintero	2015-01-03	shot and tasered	unarmed	23.0	m	hispanic	wichita	ks	False	other	flee

9. Create derived columns

```
# Create Age_in_5_Years as in the assignment.
if "Age" in df_clean.columns:
    df_clean["Age_in_5_Years"] = df_clean["Age"] + 5
    display(df_clean[["Age", "Age_in_5_Years"]].head())

# Create date-related columns for time-series analysis.
if "date" in df_clean.columns and pd.api.types.is_datetime64_any_dtype(df_clean["date"]):
    df_clean["Month"] = df_clean["date"].dt.month
    df_clean["Month_Name"] = df_clean["date"].dt.month_name()
    df_clean["YearMonth"] = df_clean["date"].dt.to_period("M").astype(str)
    display(df_clean[["date", "Year", "Month", "Month_Name", "YearMonth"]].head())
```

	Age	Age_in_5_Years
0	53.0	58.0
1	47.0	52.0
2	23.0	28.0
3	32.0	37.0
4	39.0	44.0

	date	Year	Month	Month_Name	YearMonth
0	2015-01-02	2015	1	January	2015-01
1	2015-01-02	2015	1	January	2015-01
2	2015-01-03	2015	1	January	2015-01
3	2015-01-04	2015	1	January	2015-01
4	2015-01-04	2015	1	January	2015-01

10. Descriptive statistics

```
# Select the Age column if available; otherwise use a numeric column for a general summary.
if "Age" in df_clean.columns:
    values = pd.to_numeric(df_clean["Age"], errors="coerce").dropna()
    variable_name = "Age"
else:
    numeric_available = df_clean.select_dtypes(include=np.number).columns.tolist()
    if not numeric_available:
        raise ValueError("No numeric column is available for descriptive statistics.")
    variable_name = numeric_available[0]
    values = pd.to_numeric(df_clean[variable_name], errors="coerce").dropna()

if len(values) > 0:
    summary_stats = pd.DataFrame({
        "Statistic": [
            "Count", "Mean", "Median", "Minimum", "Maximum", "Range",
            "Variance", "Standard Deviation", "Q1", "Q3", "IQR",
            "Skewness", "Kurtosis"
        ],
        "Value": [
            values.count(), values.mean(), values.median(), values.min(),
            values.max(), values.max() - values.min(), values.var(),
            values.std(), values.quantile(0.25), values.quantile(0.75),
            values.quantile(0.75) - values.quantile(0.25),
            values.skew(), values.kurt()
        ]
    })
    print("Descriptive statistics for:", variable_name)
    display(summary_stats)
```

Descriptive statistics for: Age

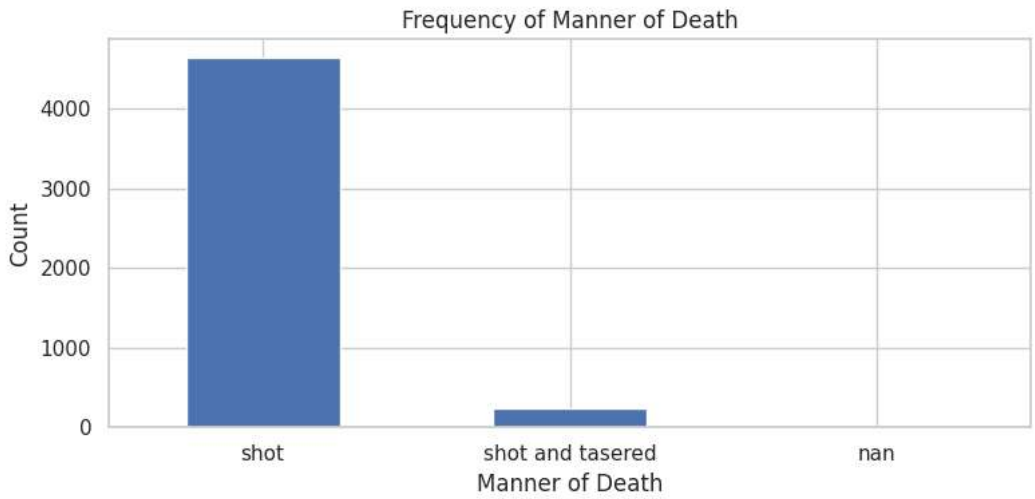
	Statistic	Value
0	Count	4895.000000
1	Mean	36.549750
2	Median	35.000000
3	Minimum	6.000000
4	Maximum	91.000000
5	Range	85.000000
6	Variance	161.146473
7	Standard Deviation	12.694348
8	Q1	27.000000
9	Q3	45.000000
10	IQR	18.000000
11	Skewness	0.714951
12	Kurtosis	0.165802

Next steps: Generate code with summary_stats | New interactive sheet

```
# Frequency distribution of manner_of_death.
if "manner_of_death" in df_clean.columns:
    manner_counts = df_clean["manner_of_death"].value_counts(dropna=False)
    display(manner_counts.rename_axis("manner_of_death").reset_index(name="count"))

    manner_counts.plot(kind="bar", figsize=(8, 4))
    plt.title("Frequency of Manner of Death")
    plt.xlabel("Manner of Death")
    plt.ylabel("Count")
    plt.xticks(rotation=0)
    plt.tight_layout()
    plt.show()
else:
    print("Column 'manner_of_death' was not found.")
```

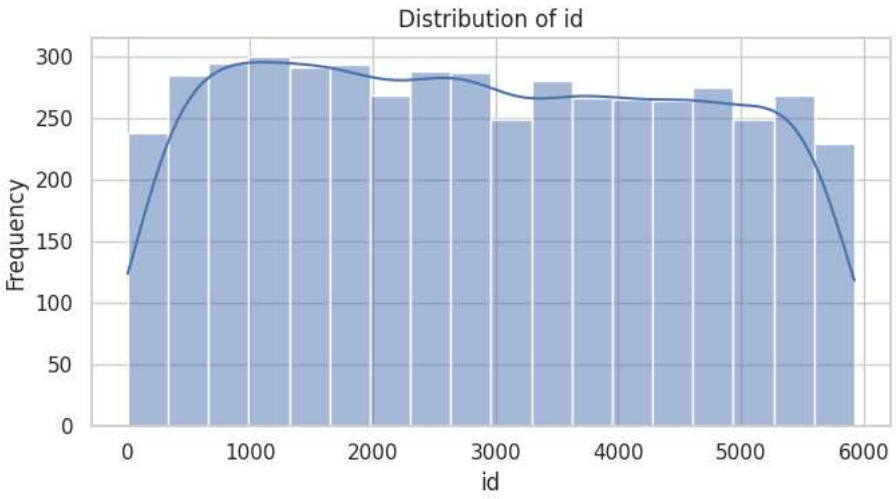
	manner_of_death	count
0	shot	4644
1	shot and tasered	248
2	NaN	3

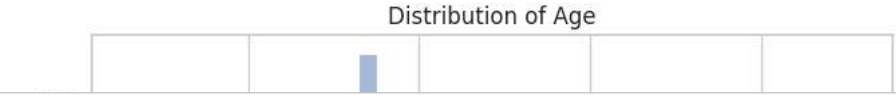


11. Univariate analysis

```
# Histograms for numeric columns.
numeric_cols = df_clean.select_dtypes(include=np.number).columns.tolist()

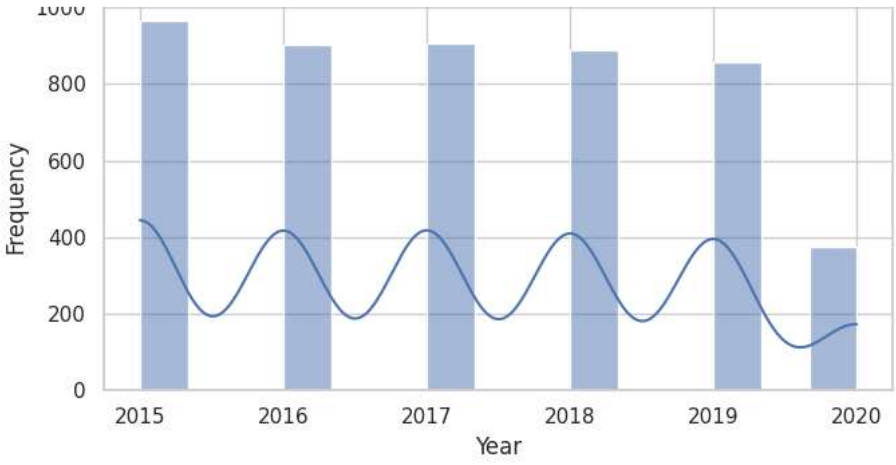
for col in numeric_cols:
    series = df_clean[col].dropna()
    if len(series) == 0:
        continue
    plt.figure(figsize=(7, 4))
    sns.histplot(series, kde=True)
    plt.title(f"Distribution of {col}")
    plt.xlabel(col)
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.show()
```

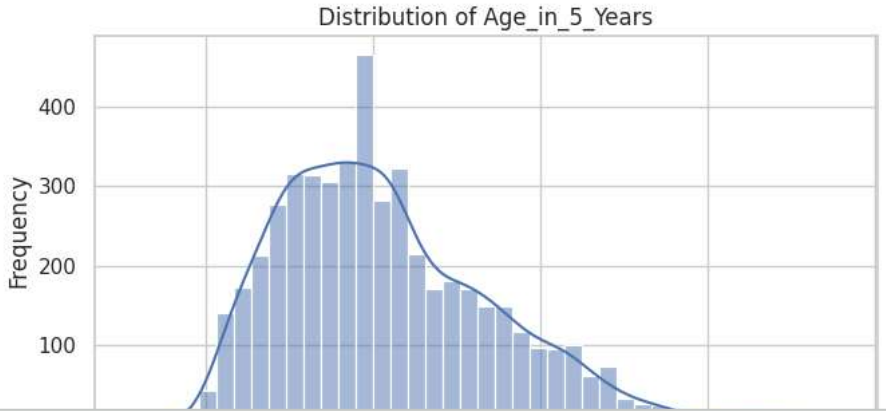


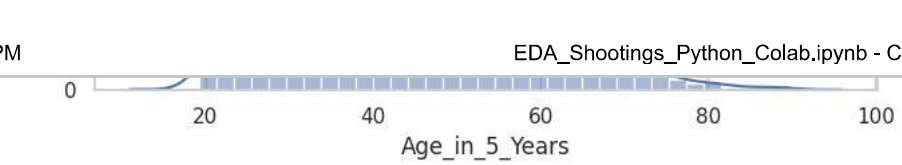


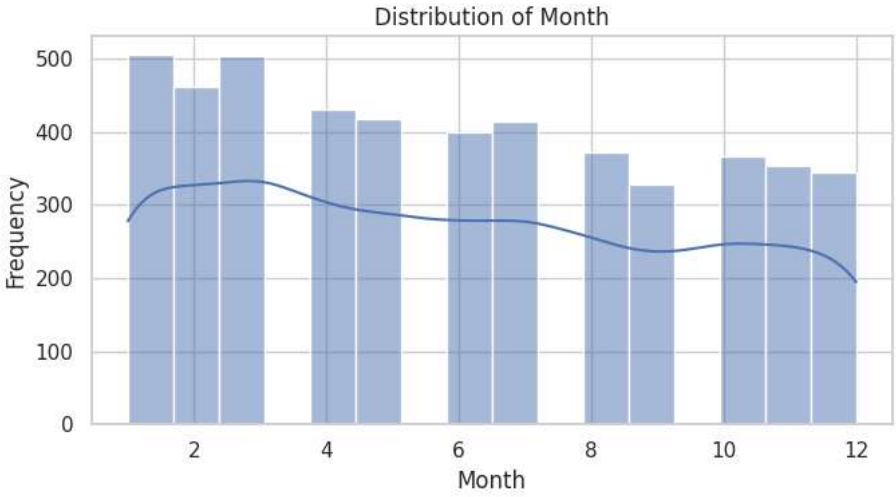

```
# Bar charts for selected categorical columns.
categorical_plot_cols = [
    "manner_of_death", "gender", "race", "threat_level",
    "flee", "body_camera", "signs_of_mental_illness"
]

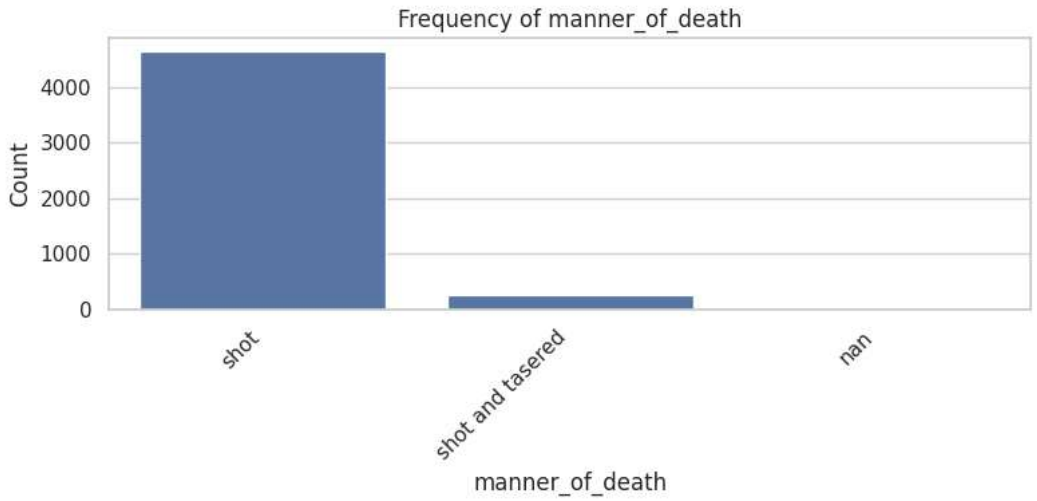
for col in categorical_plot_cols:
    if col in df_clean.columns:
        counts = df_clean[col].value_counts(dropna=False).head(15)
        plt.figure(figsize=(8, 4))
        sns.barplot(x=counts.index.astype(str), y=counts.values)
        plt.title(f"Frequency of {col}")
        plt.xlabel(col)
        plt.ylabel("Count")
        plt.xticks(rotation=45, ha="right")
        plt.tight_layout()
        plt.show()
```

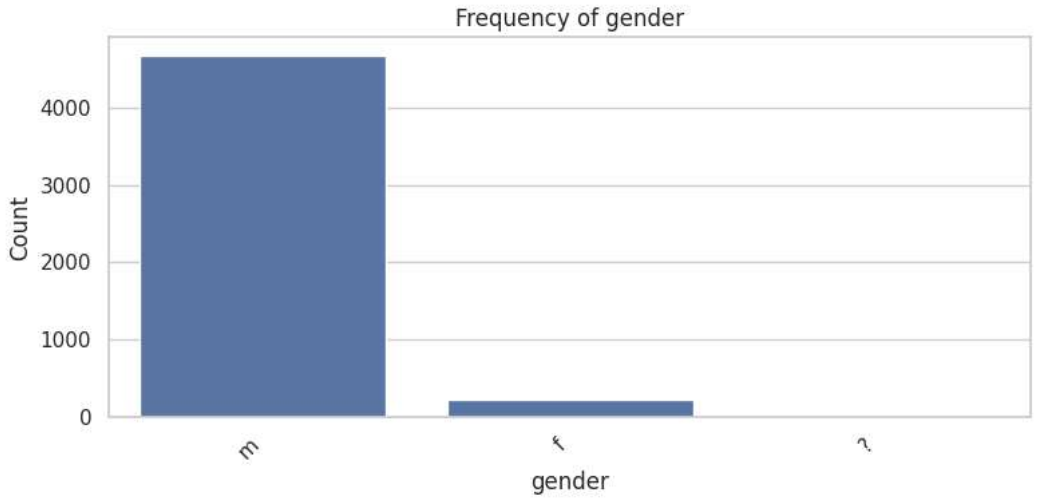


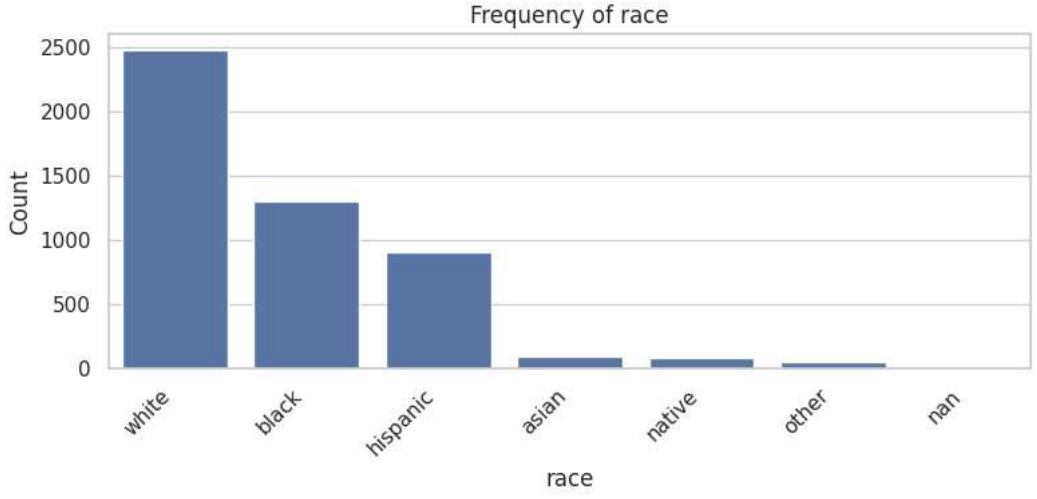


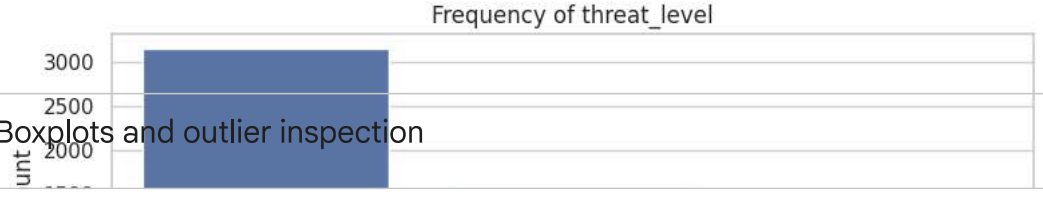

0
20
40
60
80
100
Age_in_5_Years


Distribution of Month
Frequency
500
400
300
200
100
0
2
4
6
8
10
12
Month







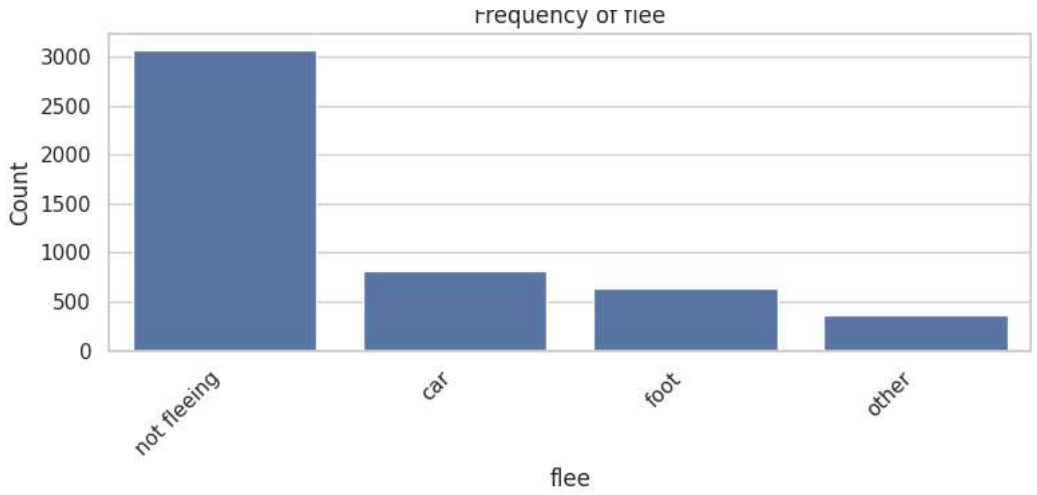


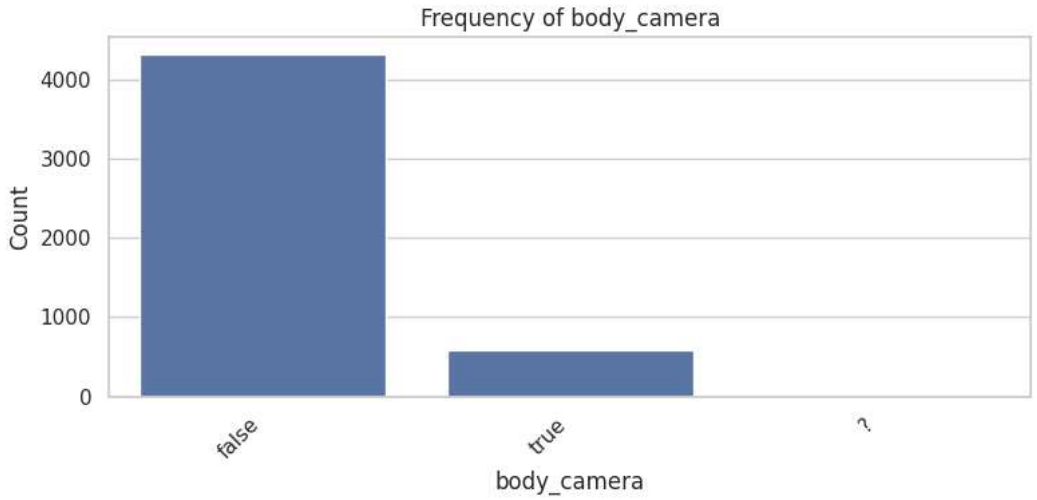
12. Boxplots and outlier inspection

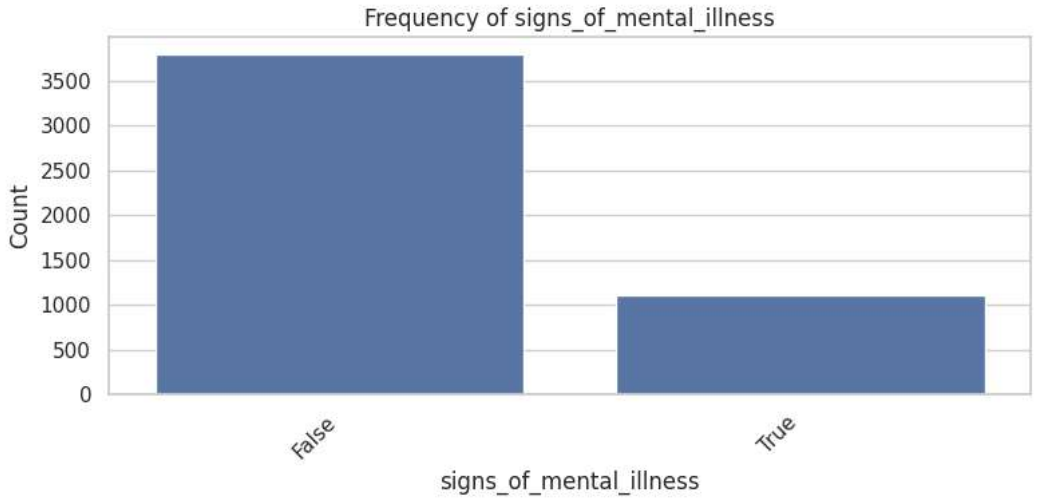
```
numeric_cols = df_clean.select_dtypes(include=np.number).columns.tolist()

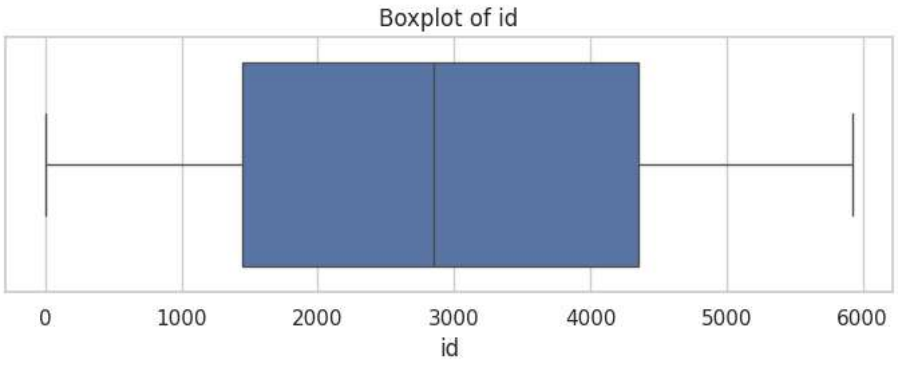
for col in numeric_cols:
    series = df_clean[col].dropna()
    if len(series) == 0:
        continue
    plt.figure(figsize=(7, 3))
    sns.boxplot(x=series)
```

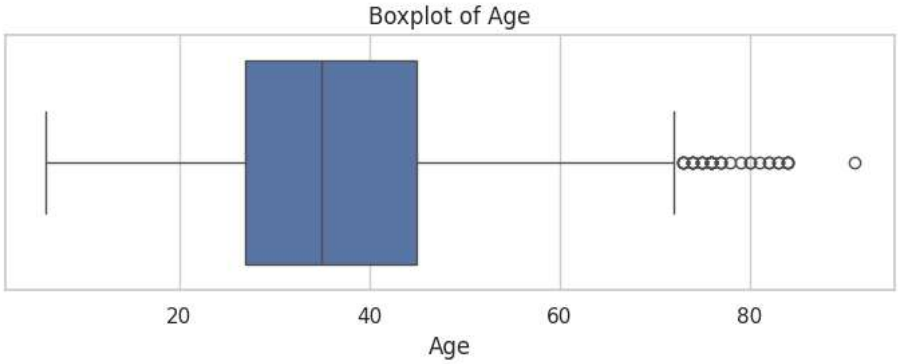
```
    plt.title(f"Boxplot of {col}")
    plt.xlabel(col)
    plt.tight_layout()
    plt.show()
```

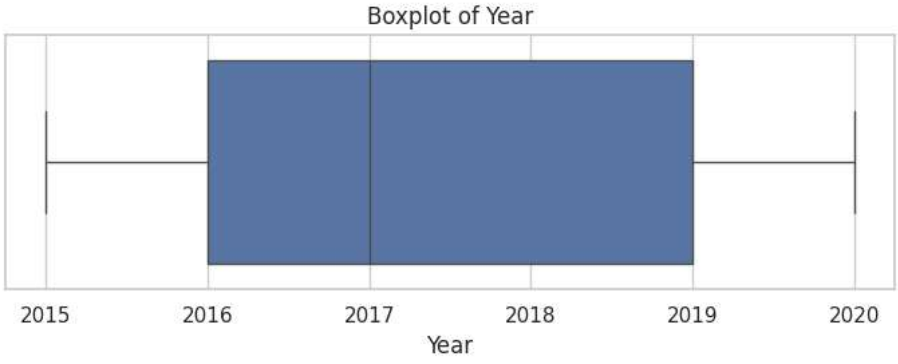


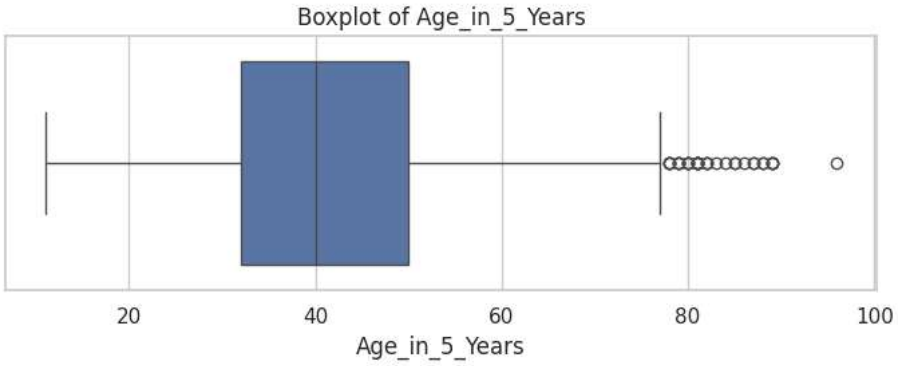


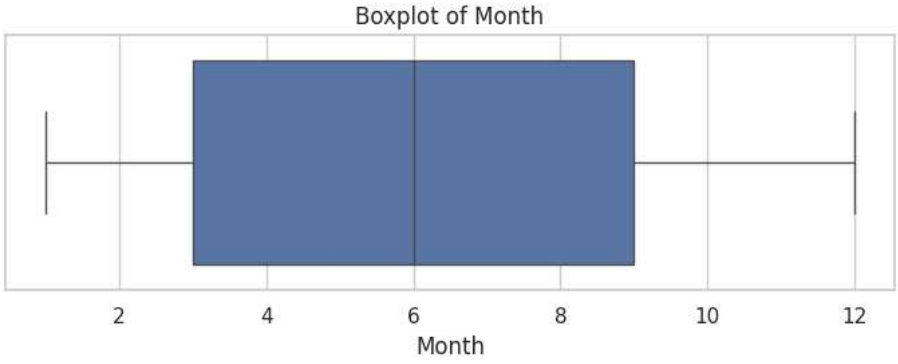



Boxplot of id
0
1000
2000
3000
4000
5000
6000
id


Boxplot of Age
20
40
60
80
Age


Boxplot of Year
2015
2016
2017
2018
2019
2020
Year


Boxplot of Age_in_5_Years
20
40
60
80
100
Age_in_5_Years


Boxplot of Month
2
4
6
8
10
12
Month

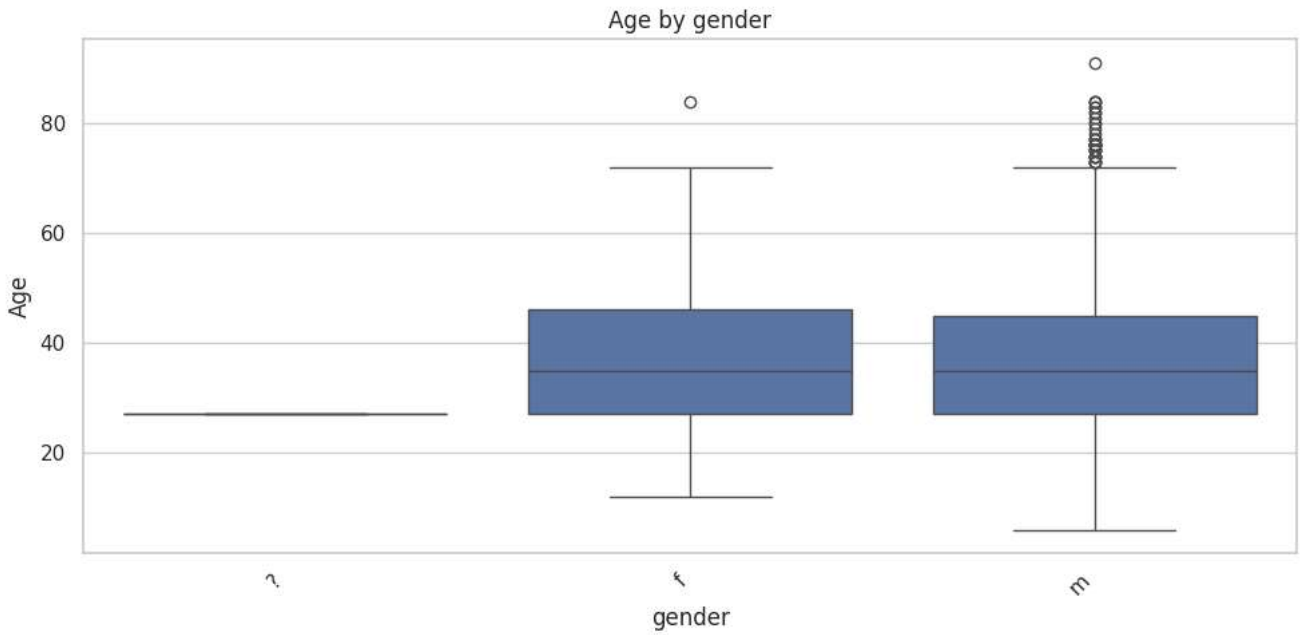
```
# IQR-based outlier counts for numeric columns.
outlier_rows = []
for col in df_clean.select_dtypes(include=np.number).columns:
    series = df_clean[col].dropna()
    if len(series) < 4:
        continue
    q1 = series.quantile(0.25)
    q3 = series.quantile(0.75)
    iqr = q3 - q1
    lower_bound = q1 - 1.5 * iqr
    upper_bound = q3 + 1.5 * iqr
    outlier_count = int(((series < lower_bound) | (series > upper_bound)).sum())
    outlier_rows.append({
        "Column": col,
        "Lower Bound": lower_bound,
        "Upper Bound": upper_bound,
        "Outlier Count": outlier_count
    })

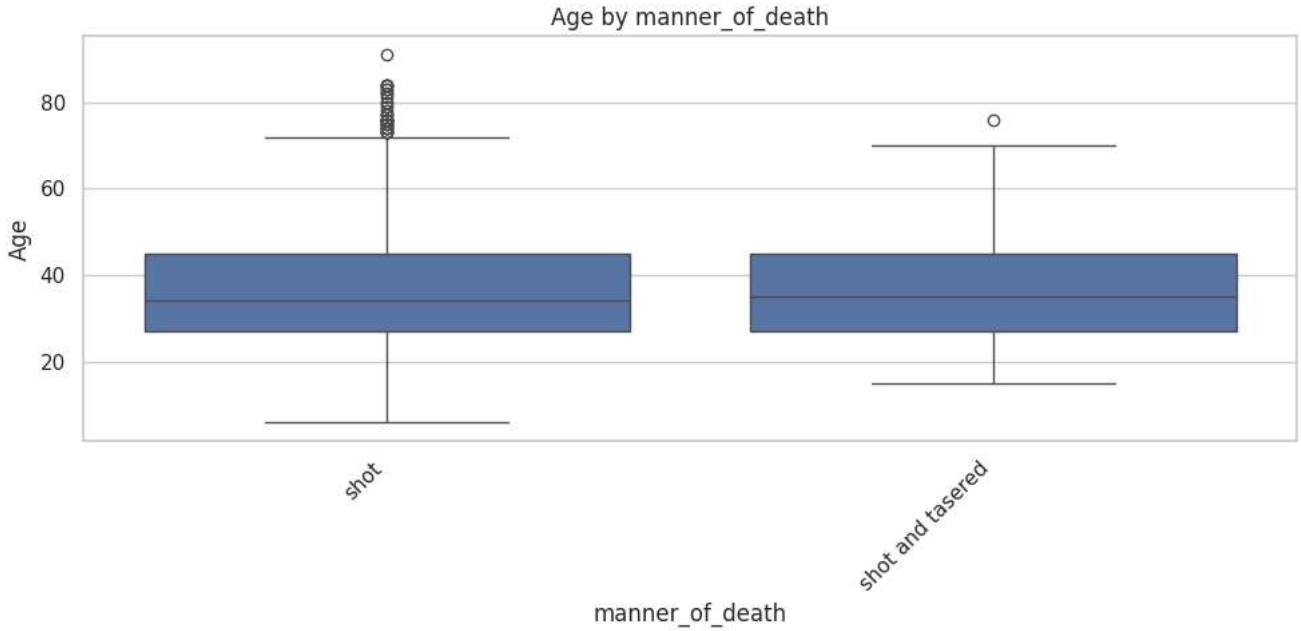
display(pd.DataFrame(outlier_rows))
```

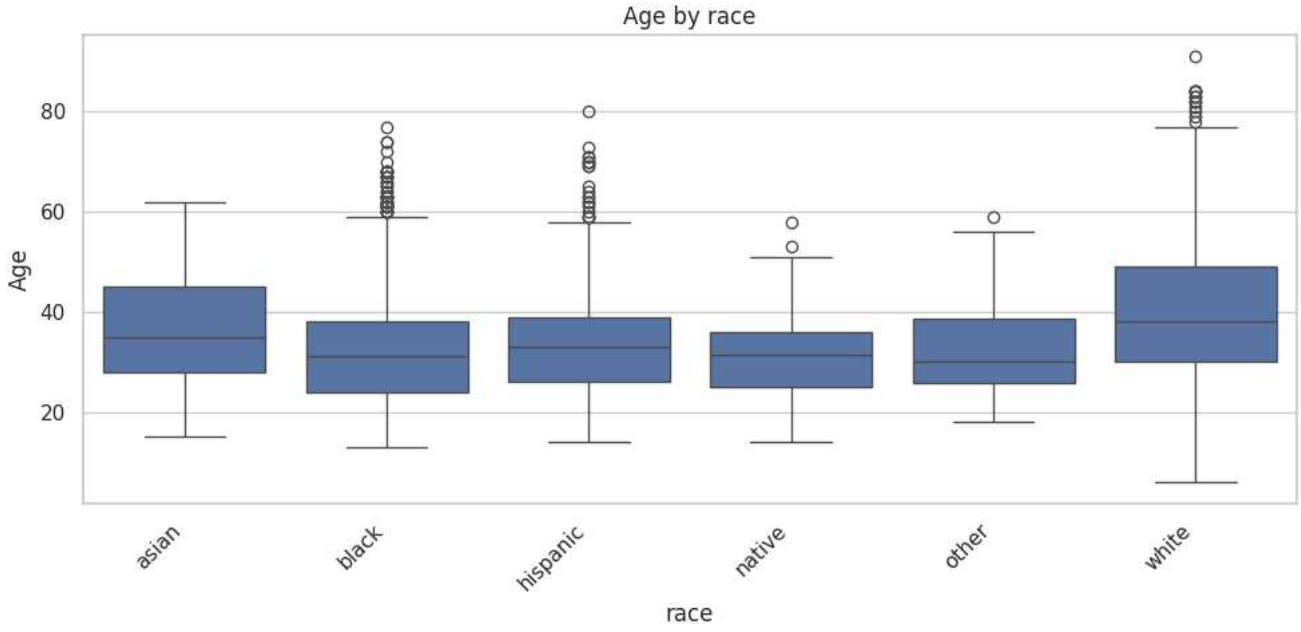
	Column	Lower Bound	Upper Bound	Outlier Count
0	id	-2925.0	8719.0	0
1	Age	0.0	72.0	37
2	Year	2011.5	2023.5	0
3	Age_in_5_Years	5.0	77.0	37
4	Month	-6.0	18.0	0

13. Bivariate and multivariate analysis

```
# Compare Age distributions across categories when those columns exist.
if "Age" in df_clean.columns:
    for cat_col in ["gender", "manner_of_death", "race"]:
        if cat_col in df_clean.columns:
            plot_data = df_clean[[cat_col, "Age"]].dropna()
            if not plot_data.empty:
                top_categories = plot_data[cat_col].value_counts().head(10).index
                plot_data = plot_data[plot_data[cat_col].isin(top_categories)]
                plt.figure(figsize=(10, 5))
                sns.boxplot(data=plot_data, x=cat_col, y="Age")
                plt.title(f"Age by {cat_col}")
                plt.xticks(rotation=45, ha="right")
                plt.tight_layout()
                plt.show()
```


Age by gender
Age
80
60
40
20
m
gender


Age by manner_of_death
Age
80
60
40
20
shot
shot and tasered
manner_of_death


Age by race
Age
80
60
40
20
asian
black
hispanic
native
other
white
race

```
# Cross-tabulation / contingency tables.
if "gender" in df_clean.columns and "manner_of_death" in df_clean.columns:
    contingency = pd.crosstab(
        df_clean["gender"], df_clean["manner_of_death"],
        margins=True
    )
    print("Contingency table: gender vs manner_of_death")
    display(contingency)

    print("Row-wise percentages:")
    display((pd.crosstab(
        df_clean["gender"], df_clean["manner_of_death"], normalize="index"
    ) * 100).round(2))
```

```
Contingency table: gender vs manner_of_death
```

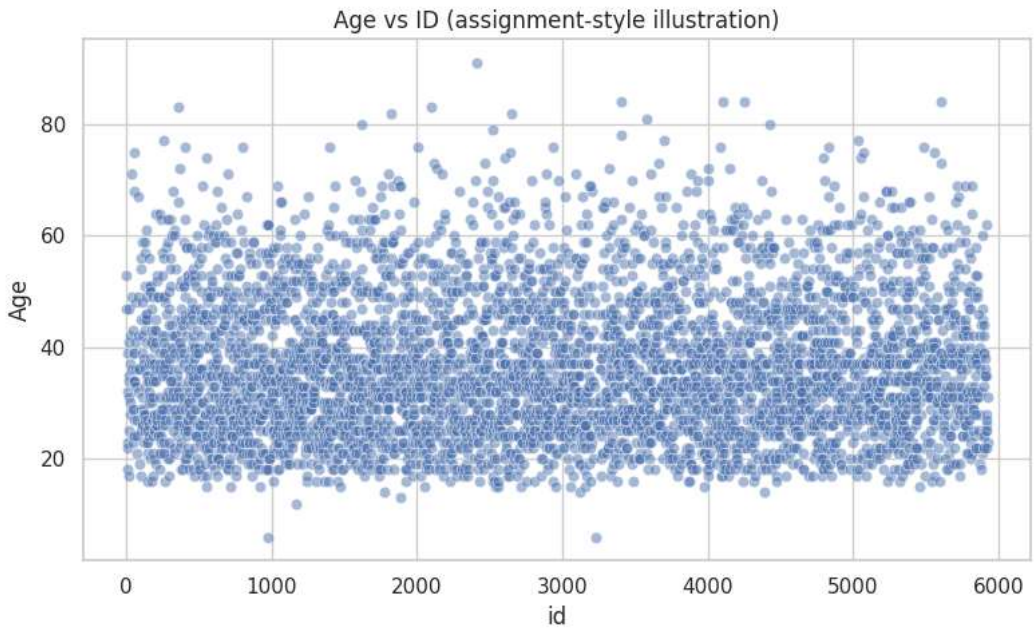
manner_of_death	shot	shot and tasered	All
gender			
?	1	0	1
f	216	6	222
m	4427	242	4669
All	4644	248	4892

```
Row-wise percentages:
```

manner_of_death	shot	shot and tasered
gender		
?	100.00	0.00
f	97.30	2.70
m	94.82	5.18

Next steps: Generate code with `contingency` New interactive sheet

```
# Scatter plot for numeric variables, if age and id are available.
# id is an identifier, so this plot is only a basic assignment-style example.
if "Age" in df_clean.columns and "id" in df_clean.columns:
    plot_data = df_clean[["id", "Age"]].dropna()
    if not plot_data.empty:
        plt.figure(figsize=(8, 5))
        sns.scatterplot(data=plot_data, x="id", y="Age", alpha=0.5)
        plt.title("Age vs ID (assignment-style illustration)")
        plt.tight_layout()
        plt.show()
```

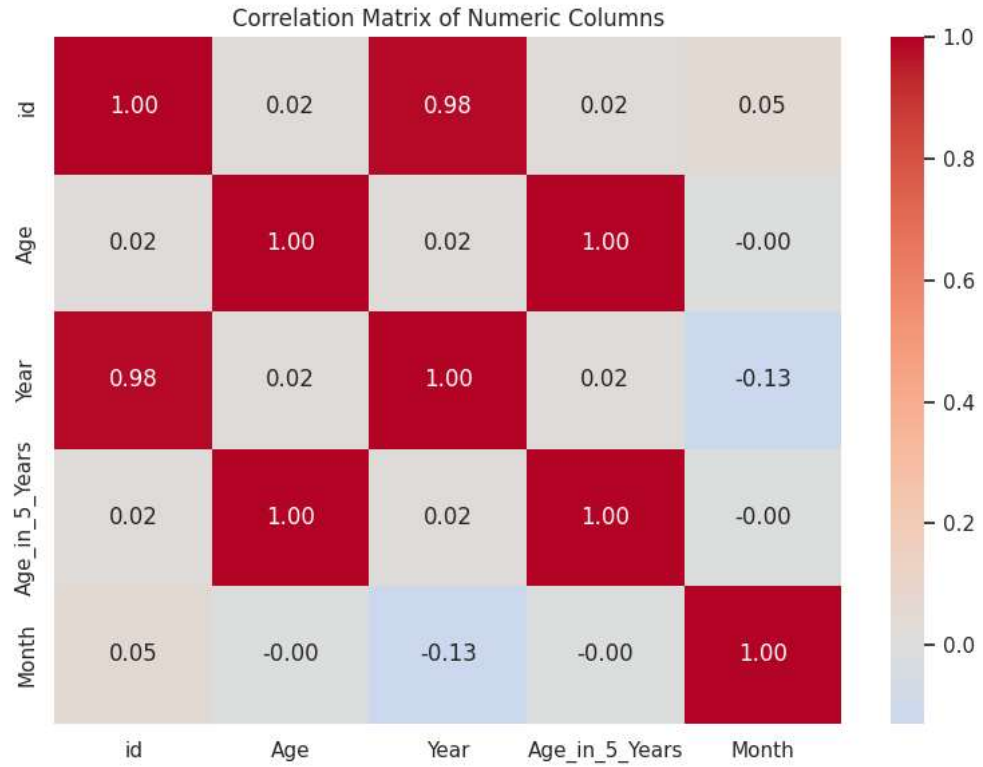


14. Correlation analysis

```
numeric_df = df_clean.select_dtypes(include=np.number)
if numeric_df.shape[1] >= 2:
    correlation_matrix = numeric_df.corr()
    display(correlation_matrix)

    plt.figure(figsize=(8, 6))
    sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", center=0, fmt=".2f")
    plt.title("Correlation Matrix of Numeric Columns")
    plt.tight_layout()
    plt.show()
else:
    print("At least two numeric columns are needed for correlation analysis.")
```

	id	Age	Year	Age_in_5_Years	Month
id	1.000000	0.022361	0.982395	0.022361	0.047948
Age	0.022361	1.000000	0.021097	1.000000	-0.001654
Year	0.982395	0.021097	1.000000	0.021097	-0.128817
Age_in_5_Years	0.022361	1.000000	0.021097	1.000000	-0.001654
Month	0.047948	-0.001654	-0.128817	-0.001654	1.000000



Next steps: Generate code with `correlation_matrix` New interactive sheet

15. Time-series analysis

```
if "date" in df_clean.columns and pd.api.types.is_datetime64_any_dtype(df_clean["date"]):
    date_df = df_clean.dropna(subset=["date"]).copy()
    date_df = date_df.set_index("date").sort_index()

    print("Date range:")
    print(date_df.index.min(), "to", date_df.index.max())

    # Number of records per month.
    monthly_counts = date_df.resample("MS").size()
    display(monthly_counts.rename("Incident Count").to_frame().head())

    plt.figure(figsize=(12, 5))
```

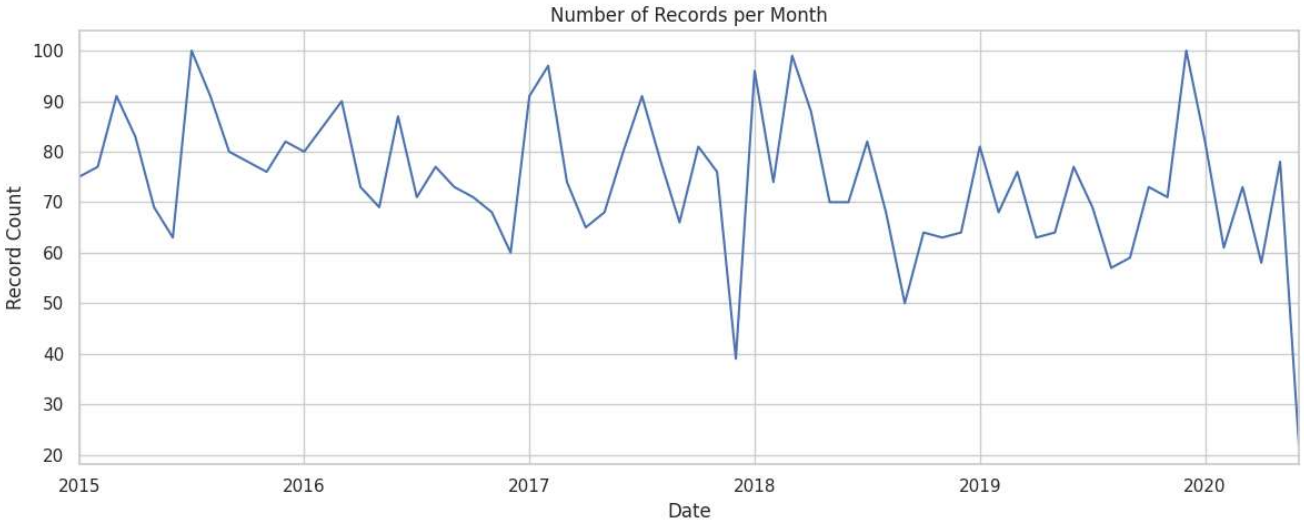
```
        monthly_counts.plot()
        plt.title("Number of Records per Month")
        plt.xlabel("Date")
        plt.ylabel("Record Count")
        plt.tight_layout()
        plt.show()

        # Number of records per year.
        yearly_counts = date_df.resample("YS").size()
        display(yearly_counts.rename("Incident Count").to_frame())

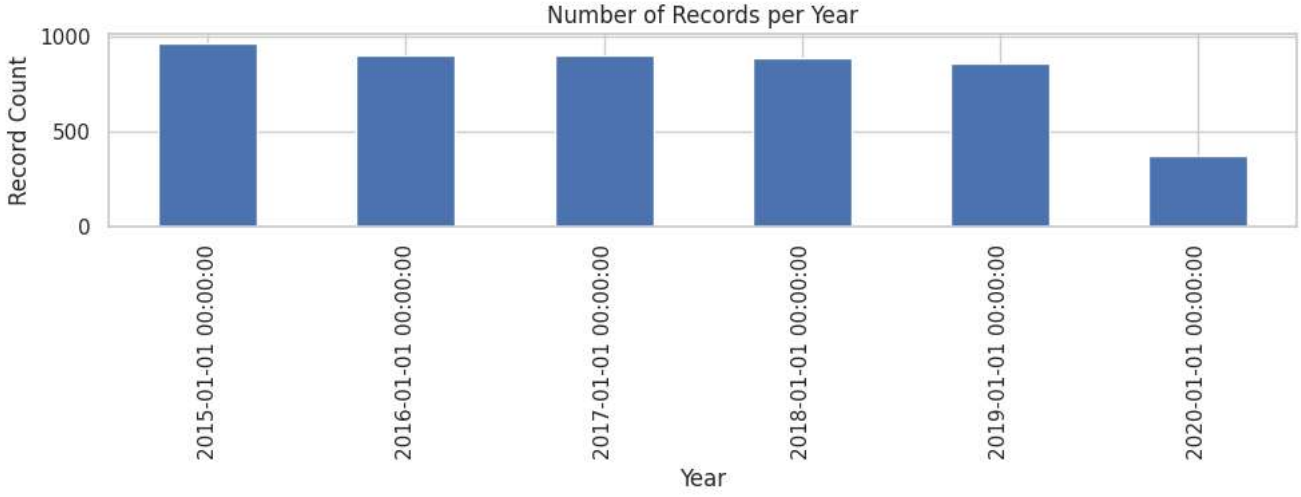
        plt.figure(figsize=(10, 4))
        yearly_counts.plot(kind="bar")
        plt.title("Number of Records per Year")
        plt.xlabel("Year")
        plt.ylabel("Record Count")
        plt.tight_layout()
        plt.show()
    else:
        print("A usable 'date' column was not found, so time-series analysis was skipped.")
```

```
Date range:
2015-01-02 00:00:00 to 2020-06-15 00:00:00
```

date	Incident Count
2015-01-01	75
2015-02-01	77
2015-03-01	91
2015-04-01	83
2015-05-01	69



date	Incident Count
2015-01-01	965
2016-01-01	904
2017-01-01	906
2018-01-01	888
2019-01-01	858
2020-01-01	374



16. K-means clustering (k = 2)

The assignment example selects `id` and `age`, scales the features, and runs K-means with two clusters and seed 123. The code below reproduces that approach when the columns exist. Because `id` is an identifier rather than a meaningful measurement, the optional alternative uses Age alone.

```
if "id" in df_clean.columns and "Age" in df_clean.columns:
    cluster_data = df_clean[["id", "Age"]].apply(pd.to_numeric, errors="coerce").dropna()

    if len(cluster_data) >= 2:
        scaler = StandardScaler()
        scaled_features = scaler.fit_transform(cluster_data)

        kmeans = KMeans(n_clusters=2, random_state=123, n_init=10)
        cluster_labels = kmeans.fit_predict(scaled_features)

        cluster_result = cluster_data.copy()
        cluster_result["Cluster"] = cluster_labels
        print("Cluster counts:")
        display(cluster_result["Cluster"].value_counts().sort_index())

        print("Cluster centers in scaled feature space:")
        display(pd.DataFrame(kmeans.cluster_centers_, columns=["id_scaled", "Age_scaled"]))

        plt.figure(figsize=(8, 5))
        sns.scatterplot(
            data=cluster_result, x="id", y="Age",
            hue="Cluster", palette="Set1", alpha=0.7
        )
        plt.title("K-means Clustering (ID and Age, k=2)")
        plt.tight_layout()
        plt.show()
    else:
        print("Not enough complete rows for K-means.")
else:
    print("Both 'id' and 'Age' are required for this assignment-style clustering example.")
```

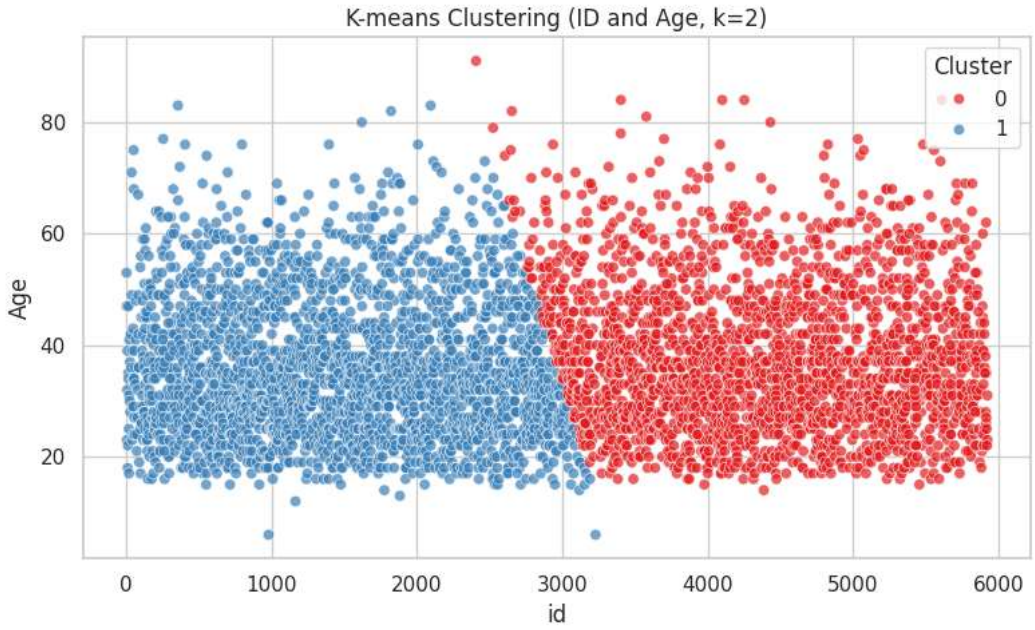
```
Cluster counts:
```

Cluster	count
0	2360
1	2535

dtype: int64

```
Cluster centers in scaled feature space:
```

	id_scaled	Age_scaled
0	0.897789	0.077637
1	-0.831037	-0.071864



16A. Optional alternative: cluster by Age only

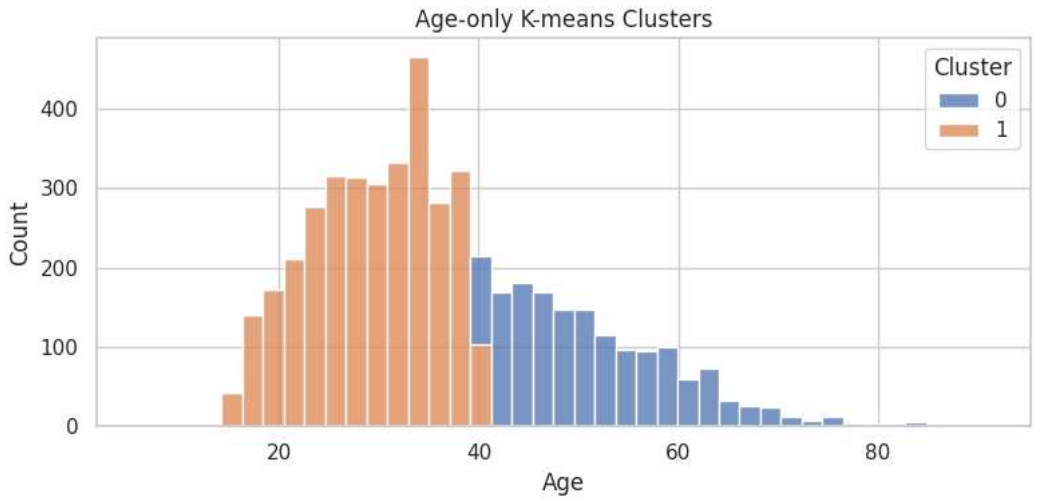
```
if "Age" in df_clean.columns:
    age_data = df_clean[["Age"]].apply(pd.to_numeric, errors="coerce").dropna()

    if len(age_data) >= 2 and age_data["Age"].nunique() >= 2:
        age_scaled = StandardScaler().fit_transform(age_data)
        age_kmeans = KMeans(n_clusters=2, random_state=123, n_init=10)
        age_data = age_data.copy()
        age_data["Cluster"] = age_kmeans.fit_predict(age_scaled)

        display(age_data.groupby("Cluster")["Age"].agg(["count", "mean", "min", "max"]))

        plt.figure(figsize=(8, 4))
        sns.histplot(data=age_data, x="Age", hue="Cluster", multiple="stack")
        plt.title("Age-only K-means Clusters")
        plt.tight_layout()
        plt.show()
```

Cluster	count	mean	min	max
0	1600	51.557500	41.0	91.0
1	3295	29.262223	6.0	40.0



17. Hierarchical clustering and dendrogram

Hierarchical clustering can be computationally expensive when applied to every row. This cell uses a sample of up to 300 rows to keep the dendrogram practical in Colab. It uses the assignment-style numeric features when available.

```
hierarchy_features = [col for col in ["id", "Age"] if col in df_clean.columns]

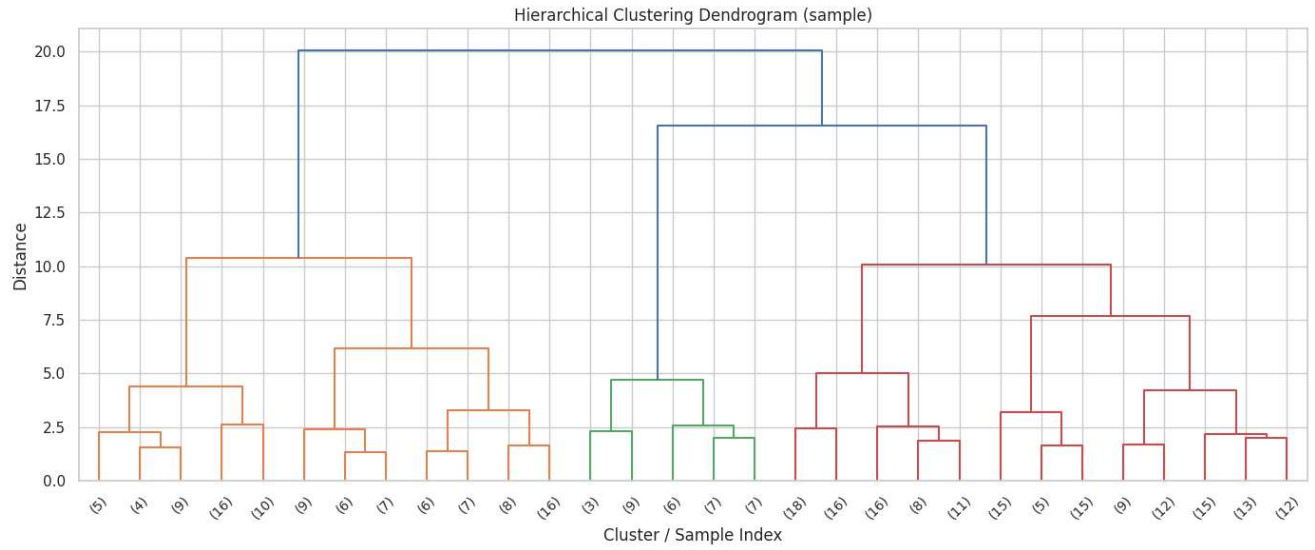
if len(hierarchy_features) >= 1:
    hierarchy_data = df_clean[hierarchy_features].apply(pd.to_numeric, errors="coerce").dropna()

    if len(hierarchy_data) >= 2:
        # Sampling keeps the dendrogram readable and limits computation.
        sample_size = min(300, len(hierarchy_data))
        hierarchy_sample = hierarchy_data.sample(n=sample_size, random_state=123)

        hierarchy_scaled = StandardScaler().fit_transform(hierarchy_sample)

        linked = linkage(hierarchy_scaled, method="ward")

        plt.figure(figsize=(14, 6))
        dendrogram(linked, truncate_mode="lastp", p=30, leaf_rotation=45, leaf_font_size=10)
        plt.title("Hierarchical Clustering Dendrogram (sample)")
        plt.xlabel("Cluster / Sample Index")
        plt.ylabel("Distance")
        plt.tight_layout()
        plt.show()
    else:
        print("Not enough complete rows for hierarchical clustering.")
else:
    print("No suitable numeric features were found.")
```



18. Agglomerative clustering labels

```
if len(hierarchy_features) >= 1:
    agg_data = df_clean[hierarchy_features].apply(pd.to_numeric, errors="coerce").dropna()

    if len(agg_data) >= 2:
        sample_size = min(1000, len(agg_data))
        agg_sample = agg_data.sample(n=sample_size, random_state=123)
        agg_scaled = StandardScaler().fit_transform(agg_sample)

        agglomerative = AgglomerativeClustering(n_clusters=2, linkage="ward")
        agg_labels = agglomerative.fit_predict(agg_scaled)

        agg_result = agg_sample.copy()
        agg_result["Cluster"] = agg_labels

        print("Agglomerative cluster counts:")
        display(agg_result["Cluster"].value_counts().sort_index())

        if "Age" in agg_result.columns:
            display(agg_result.groupby("Cluster")["Age"].agg(["count", "mean", "min", "max"]))
    else:
        print("Not enough complete rows for agglomerative clustering.")
```

```
Agglomerative cluster counts:
```

Cluster	count
0	794
1	206

dtype: int64

Cluster	count	mean	min	max
0	794	31.743376	15.0	55.0
1	206	54.543689	36.0	84.0

19. Final cleaned-data checks

```
print("Original dataset shape:", df.shape)
print("Cleaned dataset shape:", df_clean.shape)
print("Duplicate rows remaining:", int(df_clean.duplicated().sum()))
print("Missing values remaining:", int(df_clean.isna().sum().sum()))

final_missing = pd.DataFrame({
    "Missing Count": df_clean.isna().sum(),
    "Missing Percent": (df_clean.isna().mean() * 100).round(2)
}).sort_values("Missing Count", ascending=False)
display(final_missing)

display(df_clean.head())
```

```
Original dataset shape: (4895, 15)
Cleaned dataset shape: (4895, 20)
Duplicate rows remaining: 0
Missing values remaining: 7
```

	Missing Count	Missing Percent
manner_of_death	3	0.06
race	2	0.04
arms_category	2	0.04
id	0	0.00
date	0	0.00
name	0	0.00
Age	0	0.00
armed	0	0.00
city	0	0.00
state	0	0.00
signs_of_mental_illness	0	0.00
gender	0	0.00
threat_level	0	0.00
flee	0	0.00
body_camera	0	0.00
Year	0	0.00
Age_in_5_Years	0	0.00
Month	0	0.00
Month_Name	0	0.00
YearMonth	0	0.00

	id	name	date	manner_of_death	armed	Age	gender	race	city	state	signs_of_mental_illness	threat_level	f
0	3	tim elliot	2015-01-02	shot	gun	53.0	m	asian	shelton	wa	True	attack	flee
1	4	lewis lee lembke	2015-01-02	shot	gun	47.0	m	white	aloha	or	False	attack	flee
2	5	john paul quintero	2015-01-03	shot and tasered	unarmed	23.0	m	hispanic	wichita	ks	False	other	flee
3	8	matthew hoffman	2015-01-04	shot	toy weapon	32.0	m	white	san francisco	ca	True	attack	flee
4	9	michael rodriguez	2015-01-04	shot	nail gun	39.0	m	hispanic	evans	co	False	attack	flee

Next steps: Generate code with `final_missing` New interactive sheet

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```